WEEK 3 · DAY 1 · OVERVIEW

# Randomized Search and Honest Tuning




Created by **Jeff Lenamon**

ABOUT 45 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# Problem Statement

BUSINESS CONTEXT

In Course 4's capstone a colleague who covers emerging market corporates handed the desk a public file: financial statements of Polish companies, and a flag for the ones that went bankrupt within a year.

The dataset page says the bankrupt companies were analysed from 2000 to 2012 and the ones still operating from 2007 to 2013 (dataset page), so it is one country in one period.

Course 4 chose a logistic regression on it, at a fold average precision of 0.6624.

After two weeks of boosting she is back with three questions: does a booster rank these companies better, does tuning it help, and how much of a tuned score can she believe?

A model's output here is the model's probability of bankruptcy in this file, never a credit view on any company.

OBJECTIVES

Score a booster on five folds of the training rows, and count what a grid of settings costs.

Search two settings with a grid, with **`RandomizedSearchCV`** and with an Optuna study on one budget, and read a search's results fold by fold.

Measure how far reshuffling the folds alone moves a score, score a search with nested cross-validation, and choose a threshold for a recall after the search.

ABOUT 45 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

# Your Route Through Day 1

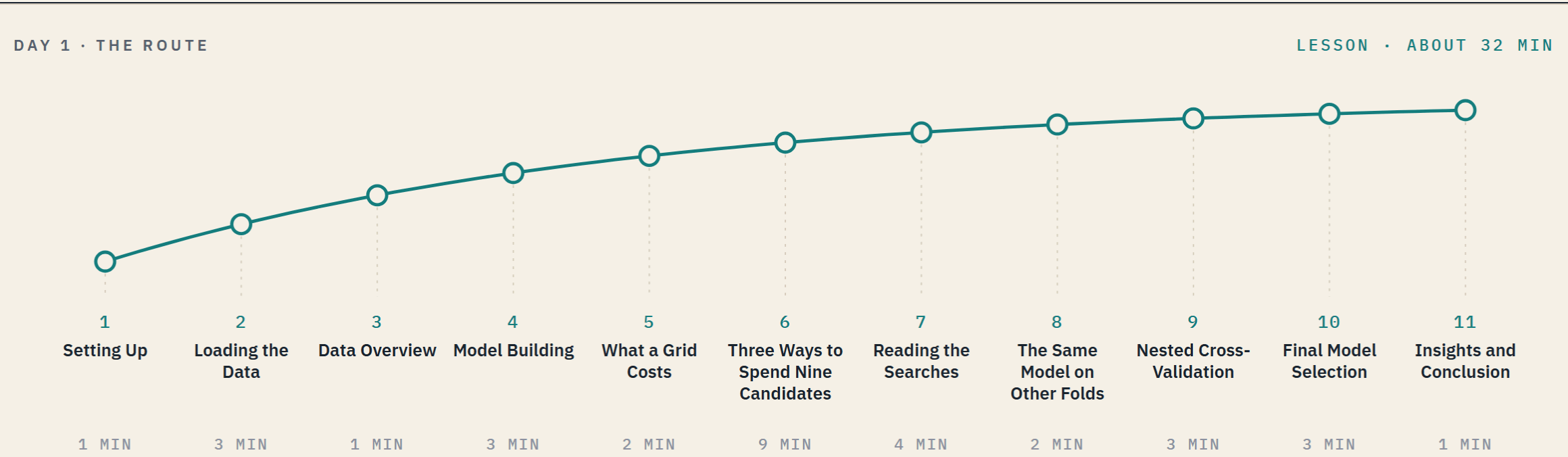



FIGURE 1 Eleven sections, in order, with about how long each takes.

**Time. About 32 minutes for the lesson, then about 12 for the four exercises.**

*Fix the seed and load the libraries.*



# Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import cross_val_predict, GridSearchCV, RandomizedSearchCV
from scipy.stats import randint, loguniform
from lightgbm import LGBMClassifier
import optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)
sns.set_theme(style="whitegrid")
sns.set_palette("colorblind")
plt.rcParams["figure.figsize"] = (10, 5)
pd.set_option("display.float_format", "{:.4f}".format)
```

**Run this cell and move on: it fixes the seed and finds the course data.**

# Loading the Data

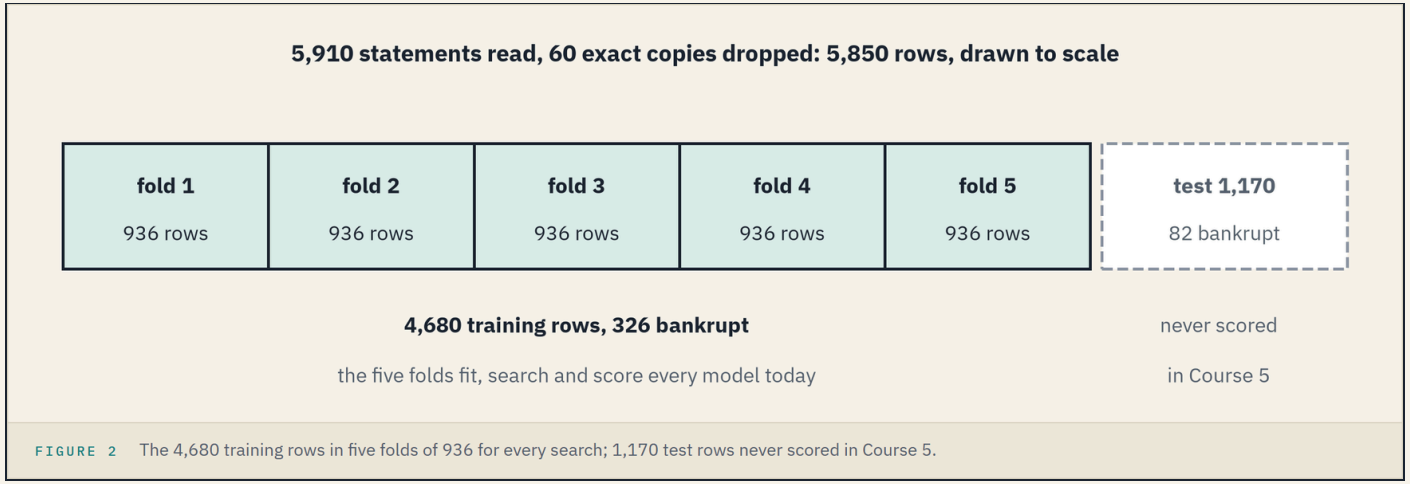



FIGURE 2 The 4,680 training rows in five folds of 936 for every search; 1,170 test rows never scored in Course 5.

### THE ROWS IT HOLDS

The same 4,680 training rows as Course 4's capstone.

The 1,170 test rows are never scored in Course 5.

### READING THE FILE

```
bankrupt = pd.read_csv(DATA + "polish_bankruptcy_5year.csv")
print(len(bankrupt), "statements,", len(FEATURES), "features")
```

```
5910 statements, 64 features
```

### COURSE 4'S ROWS

```
unique = bankrupt.drop_duplicates()
X, y = unique[FEATURES], unique[TARGET]
```

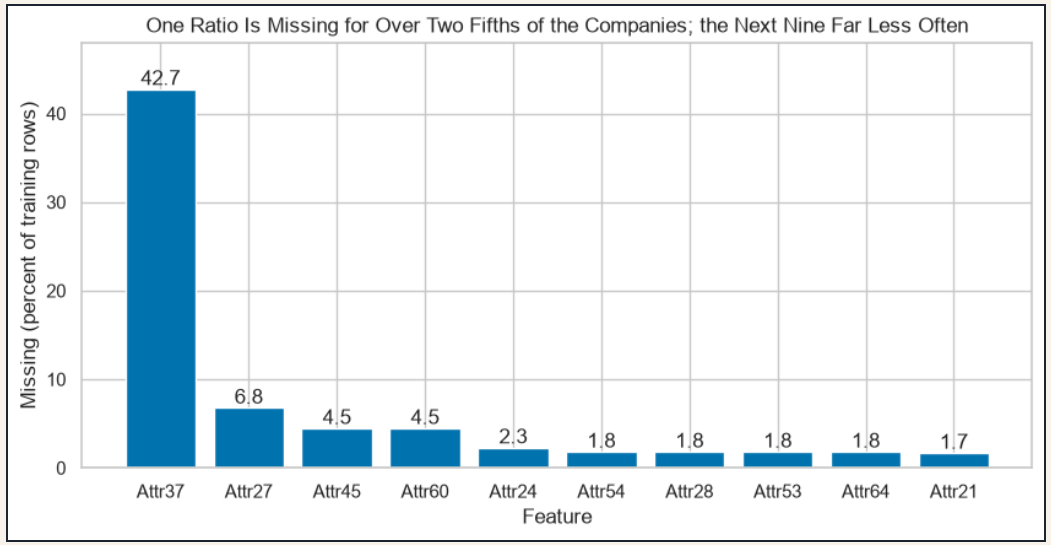
```
5850 unique rows, 408 bankrupt
Training: 4680 rows, 326 bankrupt.  Test: 1170 rows, 82 bankrupt
```

**Every search today is scored on the same five folds of the 4,680; the test rows stay closed.**

*Count the bankruptcies and the missing values.*



# Data Overview




THE RARE CLASS

With 6.97 percent bankrupt, a ranking in random order scores about 0.07 in average precision, the yardstick today.

THE GAPS

Course 4's capstone explored this file in full and filled the gaps with medians and missing indicators.

COUNTING BOTH

```
Bankrupt: 6.97 percent
Columns with a gap: 48  Companies with at least one: 2248
```

**LightGBM learns at each split which side the gaps go to, so the rows go in as they are, with no imputer.**

# Model Building

### COURSE 4'S CHOICE, FOLD BY FOLD

```
logistic_cv = cross_validate(logistic, X_train, y_train, cv=folds, scoring="average_precision", n_jobs=1)
```

```
Folds: [0.5381 0.7195 0.6186 0.675  0.7607]  Mean: 0.6624
```

### THE BOOSTER AT ITS DEFAULTS

```
default = LGBMClassifier(random_state=SEED, n_jobs=1, verbose=-1)
params = default.get_params()
print({name: params[name] for name in ["n_estimators", "num_leaves", "learning_rate", "min_child_samples"]})
```

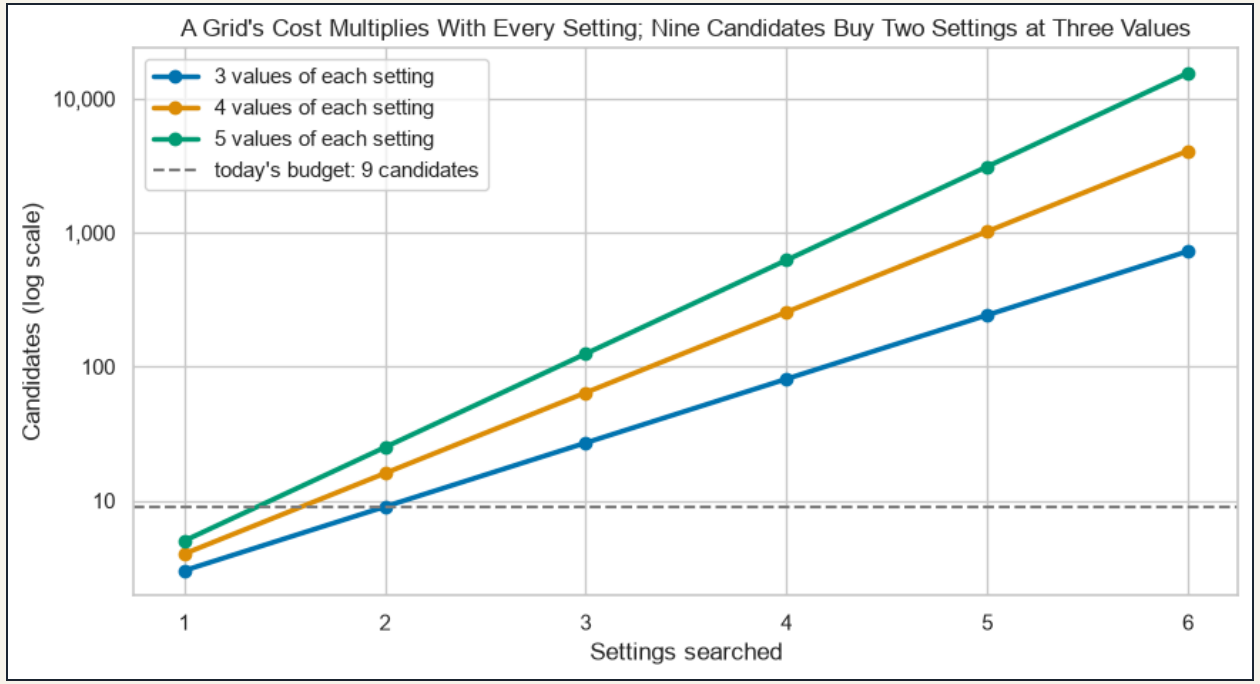
```
{'n_estimators': 100, 'num_leaves': 31, 'learning_rate': 0.1, 'min_child_samples': 20}
```

### A QUESTION TO ANSWER FIRST

Q. Day 9's **`LGBMClassifier`**, untouched, on the same five folds. Write down its fold average precision.

```
default_cv = cross_validate(default, X_train, y_train, cv=folds, scoring="average_precision", return_train_score=True, n_jobs=1)
```

# What a Grid Costs




### CANDIDATES TIMES FOLDS

```
candidates = 4 ** 6
print("Candidates:", candidates, " Fits:", candidates * 5)
```

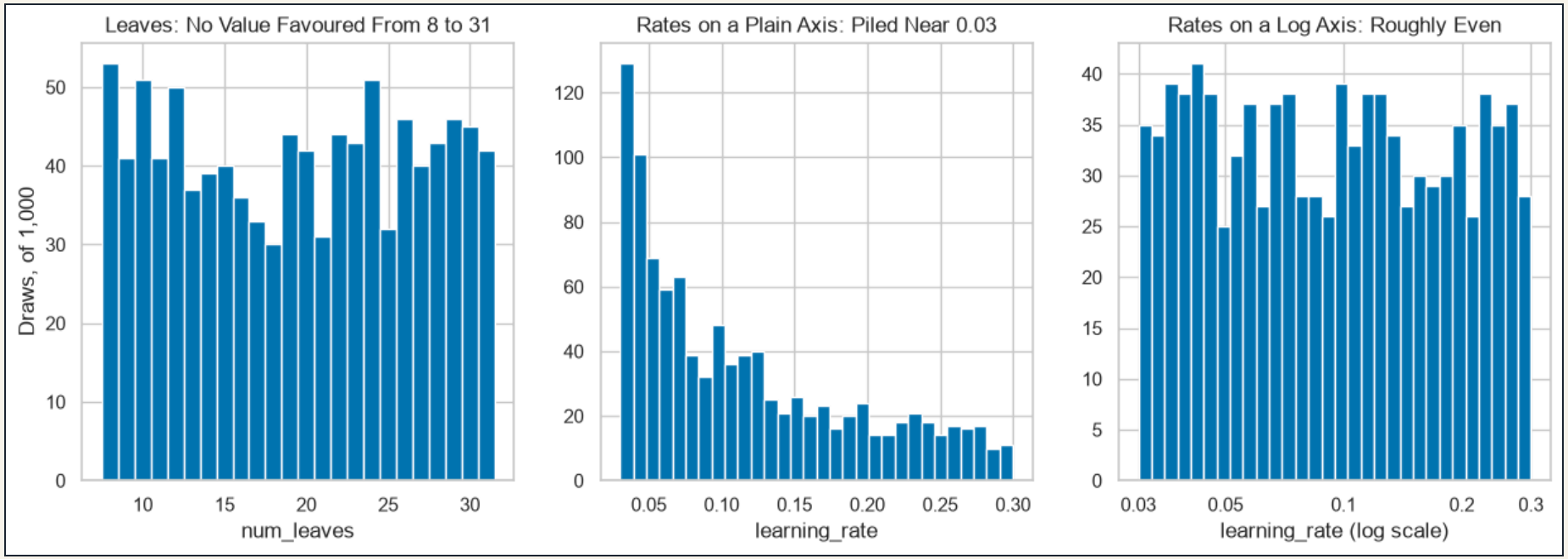
```
Candidates: 4096  Fits: 20480
```

### THE SETTING COUNT

Each setting added multiplies the candidates by its number of values.

**The budget is fixed before any search: nine candidates, 45 fits, for every search today.**

# Three Ways to Spend Nine Candidates




### A QUESTION TO ANSWER FIRST

Q. The same budget of nine candidates, drawn at random from those two distributions. Write down the best fold mean.

### A THOUSAND DRAWS FROM EACH DISTRIBUTION

```
leaf_draws = randint(8, 32).rvs(size=1000, random_state=SEED)
rate_draws = loguniform(0.03, 0.3).rvs(size=1000, random_state=SEED)

Leaves: 8 to 31  Rates below 0.1: 530 of 1000
```

# Reading the Searches

THE RUNNING BEST

`np.maximum.accumulate(values)` keeps, at each position, the largest value up to there: the **running best**.

THE SPREAD OF FOLDS AGAINST THE SPREAD OF MEANS

```
fold_scores = pd.DataFrame({"fold " + str(k + 1): search.cv_results_["split" + str(k) + "_test_score"] for k in range(5)}, index=range(1, 10))
spans = fold_scores.max(axis=1) - fold_scores.min(axis=1)
print("The nine means span:", round(results["mean"].max() - results["mean"].min(), 4))
print("One candidate's five folds span:", round(spans.min(), 4), "to", round(spans.max(), 4))
```

```
The nine means span: 0.0246
One candidate's five folds span: 0.1209 to 0.1915
```

**The rows in a fold set the level; the settings move it a little. Next we measure how far the folds alone move a score.**

# The Same Model on Other Folds

FIVE FOLDS, SHUFFLED BY SEED

```
folds = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
```

A QUESTION TO ANSWER FIRST

Q. The untuned defaults, scored on eight shuffles of the folds, **`random_state=SEED + shift`** for **`shift`** from 0 to 7. Write down the best of the eight means.

EIGHT SHUFFLES OF THE FOLDS

A fold split is one shuffle of the rows; another **`random_state`** gives another five folds of the same 4,680.

```
shuffled = {}
for shift in range(8):
    reshuffled = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED + shift)
    shuffled[SEED + shift] = cross_val_score(default, X_train, y_train, cv=reshuffled, scoring="average_precision", n_jobs=1).mean()
shuffled = pd.Series(shuffled)
print("Eight means:", round(shuffled.min(), 4), "to", round(shuffled.max(), 4), " Best at seed", shuffled.idxmax())
```

*Run the whole search inside each outer fold.*



# Nested Cross-Validation

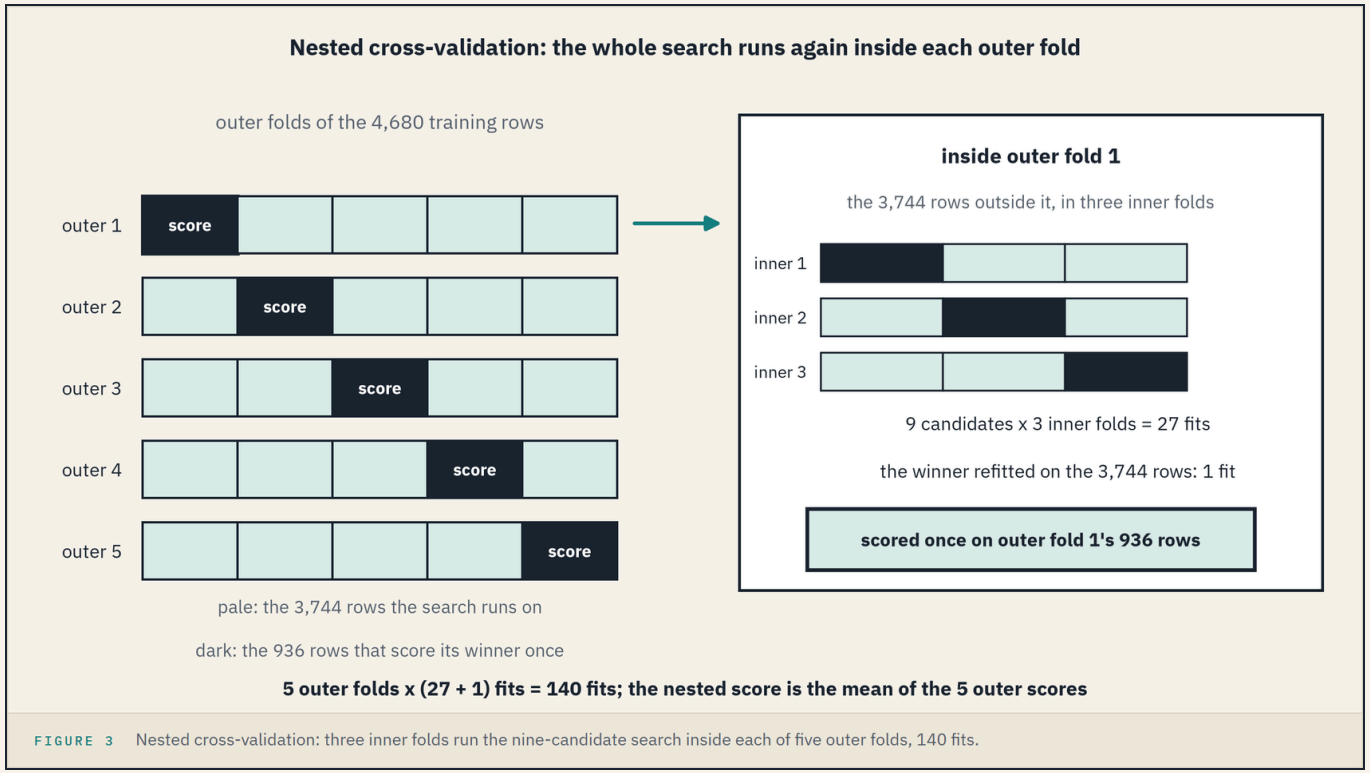



FIGURE 3 Nested cross-validation: three inner folds run the nine-candidate search inside each of five outer folds, 140 fits.

### A QUESTION TO ANSWER FIRST

Q. Score the same search by nested cross-validation. Write down its nested mean.

### SCORING A SEARCH

**Nested cross-validation** scores a search, not a candidate.

### THE INNER FOLDS AND THE SEARCH

```
inner_search = RandomizedSearchCV(LGBMClassifier(random_state=SEED, n_jobs=1, verbose=-1), space, n_iter=9,
                                  cv=inner, scoring="average_precision", random_state=SEED, n_jobs=1)
```

# Final Model Selection

EACH THRESHOLD AT A RECALL OF 0.70

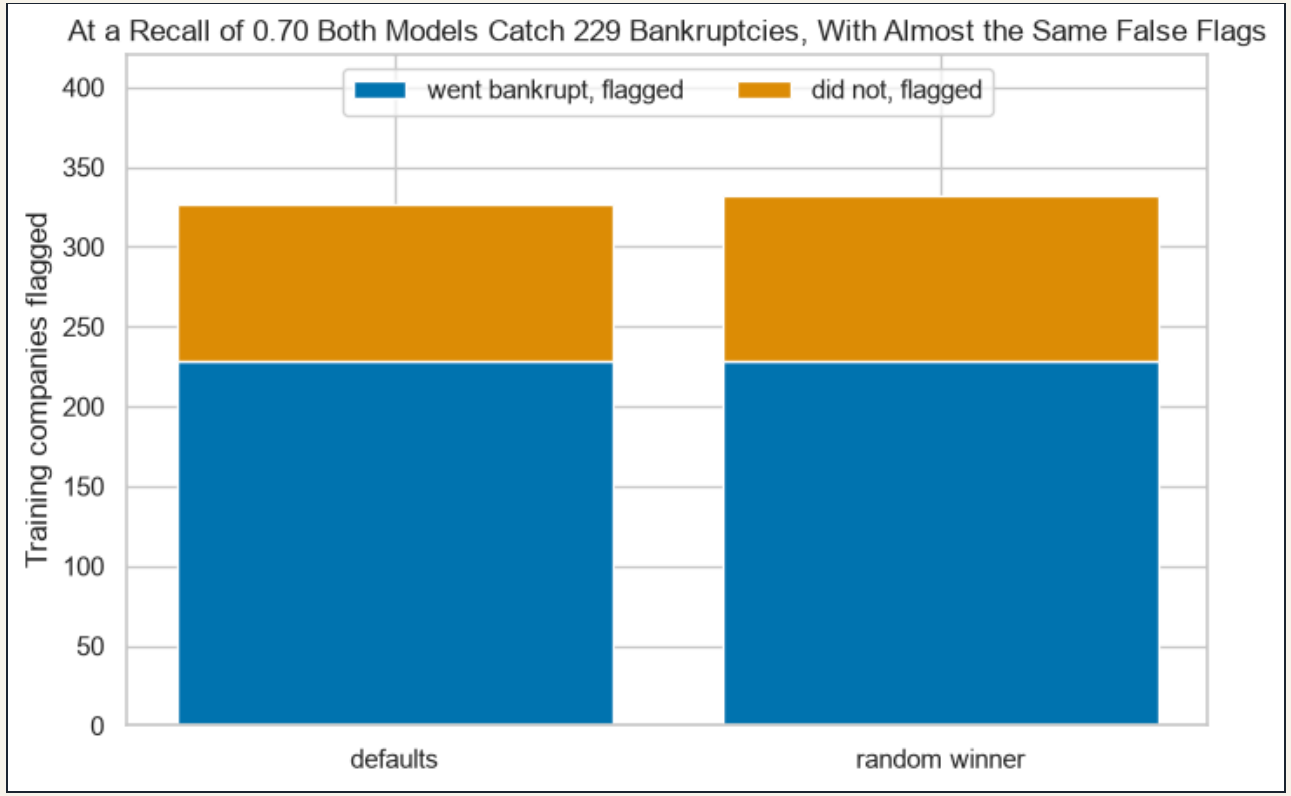
```
for model in [default, search.best_estimator_]:
    out_of_fold = cross_val_predict(model, X_train, y_train, cv=folds, method="predict_proba", n_jobs=1)[:, 1]
    precision, recall, thresholds = precision_recall_curve(y_train, out_of_fold)
    threshold = thresholds[recall[:-1] >= 0.70].max()
```

FLAGS AT EACH THRESHOLD

| | THRESHOLD | FLAGGED | CAUGHT | PRECISION |
|---|---|---|---|---|
| defaults | 0.0818 | 327 | 229 | 0.7003 |
| random winner | 0.0333 | 332 | 229 | 0.6898 |

**A threshold is a setting like any other, chosen here on the training rows' out-of-fold probabilities.**

# Insights and Conclusion




WHAT THE BARS SHOW

At them both catch 229 of the 326 bankruptcies, and the defaults flag 5 fewer companies.

The thresholds differ, 0.0818 and 0.0333, because the two models spread their probabilities differently.

**In practice: fix the budget and the folds before looking, search a rate on a log scale, stop when the running best is flat, and report the nested score beside `best_score_`.**

# You Can Now

- Count what a grid of settings costs, and fix a search's budget before it runs.
- Search a space with **`RandomizedSearchCV`** and **`scipy.stats`** distributions, and with an Optuna study.
- Read **`cv_results_`** fold by fold, and draw a search's running best.
- Measure how far reshuffling the folds moves a score.
- Score a search with nested cross-validation, and choose a threshold after it.

NEXT
WEEK 3 · DAY 2

## Feature Engineering on Financial Ratios

Day 12 takes the defaults forward and builds new columns from the ratios.